Citexis: A Knowledge Discovery System for Research Papers

Project Report and Documentation

Submitted By:

Suraj Karki
Matrikel-Nr: 50178465

Deepti Hegde Mahabaleshwar
Matrikel-Nr: 50298233

To: Prof. Dr. rer. nat. Alexander Maier

Prepared for the Data Science (M.Sc.) program at
Bielefeld University of Applied Sciences



Department of Engineering and Mathematics

Hochschule Bielefeld, Germany

23.07.2026

Contents

1 Introduction **1**
- 1.1 Problem Statement . . . 1
- 1.2 Project Objectives . . . 1

2 System Design and Architecture **2**
- 2.1 Architectural Overview . . . 2
- 2.2 Technology Stack . . . 2
  - 2.2.1 React.js . . . 3
  - 2.2.2 FastAPI . . . 3
  - 2.2.3 MongoDB . . . 3
  - 2.2.4 Elasticsearch . . . 3
  - 2.2.5 Pinecone . . . 4
  - 2.2.6 Celery and Redis . . . 4
  - 2.2.7 Docker and Docker Compose . . . 4
- 2.3 Client Layer . . . 4
- 2.4 API Layer . . . 4
- 2.5 Service Layer . . . 6
- 2.6 Data Layer . . . 6
- 2.7 Background Processing . . . 8

3 Implementation **8**
- 3.1 Ingestion Pipeline . . . 9
- 3.2 Hybrid Search . . . 9
- 3.3 Recommendation Engine . . . 11
- 3.4 Authentication and Access Control . . . 13

4 Experimental Setup **13**
- 4.1 Embedding Model . . . 13
- 4.2 Ranking Model . . . 13
- 4.3 Hybrid Fusion Parameters . . . 14

5 Discussion **14**
- 5.1 Technical Challenges and Solutions . . . 14
- 5.2 Evaluation and Reflection . . . 14
- 5.3 Limitations . . . 15

6 Conclusion and Future Work **15**

References **16**

A Setup and Installation Guide **17**
- A.1 Prerequisites . . . 17
- A.2 Environment Configuration . . . 17
- A.3 Starting the Application . . . 18
- A.4 Verifying the Deployment . . . 19
- A.5 Index Maintenance . . . 19

B User Interface Walkthrough **20**

1 Introduction

Due to increasing numbers of publications, open-access repositories, and preprint servers, the size of academic literature has expanded rapidly across nearly every discipline. Large-scale literature databases such as OpenAlex [7], Crossref [2], and general-purpose academic search platforms have made it possible to index hundreds of millions of records, and tools such as Google Scholar have become widely used entry points for literature discovery. Mostly, these platforms use keyword and citation-based retrieval techniques such as BM25 [10].

The popularity of semantic-based search in academic research tools has increased because it finds similar research literature based on content and meaning rather than keywords or exact sentence matches. In contrast, when recommending related works, citation-based metrics such as citation counts are widely used because of their influence. Recommender systems commonly combine different types of information, such as content similarity, relationships between items, and user preferences, to provide more relevant recommendations [9]. The main motivation behind the Citexis project is that keyword search, semantic search, and citation-based metrics each capture different and complementary aspects of relevance and combining these approaches through a fusion method can therefore improve search and recommendation quality.

One such fusion method is Reciprocal Rank Fusion [1], which merges different rankers based on document rankings rather than scores. Therefore, Citexis uses a polyglot architecture, where different databases are used for different purposes and managed through a central orchestration layer.

1.1 Problem Statement

There are several challenges that arise in the discovery of academic literature, which are summarized in Table 1.

Challenge	Description
Research volume	The increasing number of research papers makes a systematic literature review difficult.
Keyword search limitations	Searching with exact terms cannot find papers that use relevant concepts but different wording.
Semantic search misalignment	Semantic search may provide papers that appear similar in meaning but are not related in purpose.
Citation vs. relevance	A highly cited paper may still not match the required topic or context.
Discovery beyond search	Users need recommendations, topic exploration, and reading lists rather than only query-based retrieval.

Table 1: Challenges in the problem domain

1.2 Project Objectives

Primary Objective

> **To design and implement a system for exploring and discovering relationships between research papers.**

Sub-tasks

The following sub-tasks are performed to achieve the main objective:

1. Store and manage research paper metadata efficiently.
2. Generate vector embeddings for semantic-based retrieval.
3. Add support for both keyword-based and semantic search.
4. Implement a hybrid search technique which combines semantic and keyword search.
5. Add a feature that recommends papers related to the user's search or the selected paper.
6. Develop a full-fledged web application for the research discovery platform.

2 System Design and Architecture

This section presents the system architecture of Citexis. The system implements a layered architecture in which presentation, application control, business logic, data access, and infrastructure are divided into distinct tiers.

2.1 Architectural Overview

Layer	Responsibility
Client	User interface of the Citexis
API	Routing, data validation, authentication, and task dispatch
Service	Business logic: search, recommendations, ingestion, authentication, indexing, and explore
Data	Durable storage: metadata, indexes, and user data

Table 2: Architectural layer summary.

As shown in Figure 1, the architecture has five cooperating layers. At the top, the Client Layer exposes the user interface through both a web application and a Command-Line Interface (CLI), which provides users with two ways to interact with the system. All client requests pass through the API Layer, where incoming REST API calls are routed and validated before being handed off via dependency injection to the appropriate service. The Service Layer forms the core of the system, containing the ingestion pipeline, the search and explore functionality, the authentication system, the recommendation engine, and the indexing logic. This layer manages the application logic and is the only layer allowed to communicate with multiple databases. It uses a repository abstraction to access the Data Layer, hiding storage details from the rest of the application. The Data Layer provides persistent storage and retrieval services for the application data. Background processing components handle long-running tasks asynchronously without blocking API requests.

2.2 Technology Stack

The system uses a modern full-stack technologies designed to support scalable literature discovery, efficient search, recommendation, and reproducible deployment.

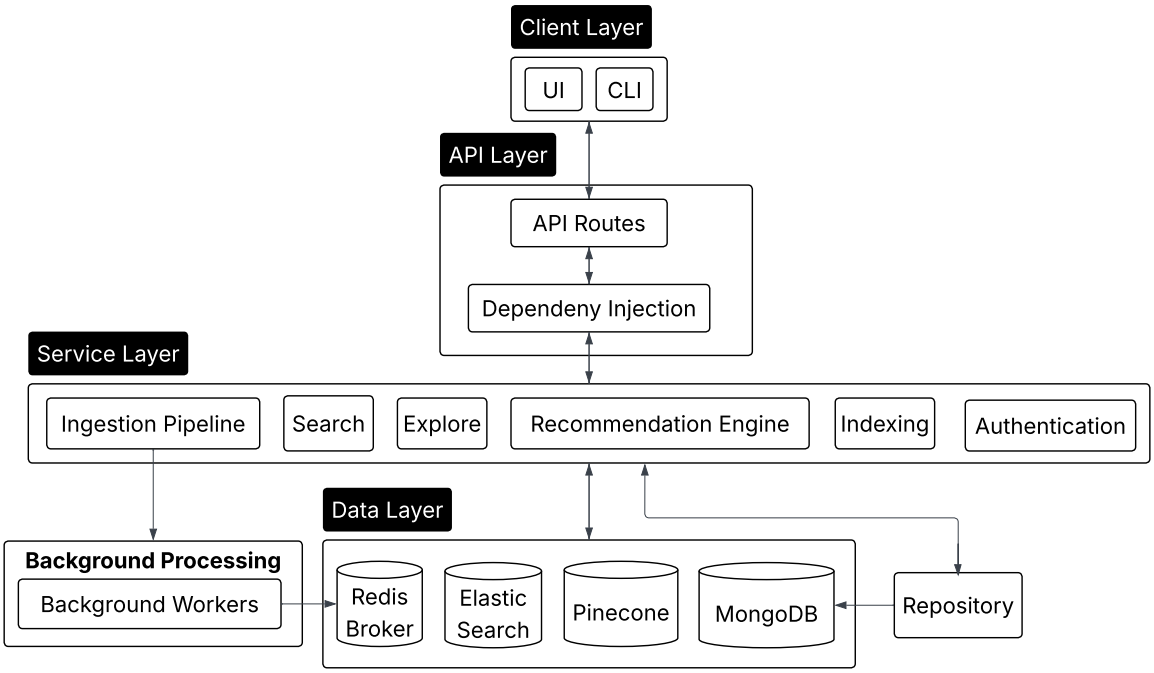


Figure 1: Citexis layered system architecture

2.2.1 React.js

React is a free and open-source JavaScript library for building component-based user interfaces [8]. It is used to develop the interactive web interface of Citexis, providing reusable components and efficient state and UI management across the pages.

2.2.2 FastAPI

FastAPI is a modern, high-performance Python web framework for developing RESTful APIs using standard Python type annotations [4]. In this system, FastAPI serves as the backend API framework, enabling efficient request processing, URL routing, and dependency management. Its asynchronous support and automatic API documentation suit a Python REST backend.

2.2.3 MongoDB

MongoDB is used as the primary document database for storing application data, including paper metadata, users, and interactions [5]. Its document model stores paper metadata with varying fields, such as authors, tags, and citations, without a fixed schema.

2.2.4 Elasticsearch

Elasticsearch provides keyword-based retrieval using BM25 ranking and metadata filtering through indexed documents [3].

2.2.5 Pinecone

Pinecone is used for storing paper embeddings and enabling semantic similarity search through vector retrieval [6]. It is a managed vector database, so semantic search works without hosting a separate vector index.

2.2.6 Celery and Redis

Celery is used for asynchronous task execution, which allows long-running operations such as bulk paper ingestion and index updates to run independently of API requests. Redis acts as the message broker, which manages communication between the application and background workers.

2.2.7 Docker and Docker Compose

Docker is used to containerize the application components, ensuring a consistent runtime environment. Docker Compose manages the deployment and communication of multiple services, including the frontend, backend, databases, and background workers.

2.3 Client Layer

The client layer is the uppermost tier of the system and provides the web interface through which researchers search, browse, and organise research papers, and through which administrators manage paper imports. It is implemented as a single-page application in React and communicates exclusively with the FastAPI backend using a REST API over HTTP. All data access goes through the API layer.

Authentication is implemented using JWT-based access control. After login or registration, the access token and user profile are stored in the browser's local storage and attached to subsequent requests via an `Authorization: Bearer` header. Route guards restrict `/saved` to authenticated users and `/import` to administrators. Figure 2 shows the client screens and their relationship to the API. Table 3 summarizes each module, its route, and its primary backend interaction.

UI module	Route	Primary API interaction
Search	`/`	`POST /search`
Paper detail	`/papers/:id`	`GET /papers/{id}` + interactions
Related papers	`/papers/:id/related`	`GET /papers/{id}/related`
Author	`/authors/:name`	`GET /authors/{name}/papers`
Explore	`/explore`	`GET /explore/topics`
Saved papers	`/saved`	`GET /interactions/me/saved`
Import	`/import`	`POST /papers/import/*` (admin)
Authentication	`/login`	`POST /auth/login`
Register	`/register`	`POST /auth/register`

Table 3: Client modules and their API interactions.

2.4 API Layer

The API layer acts as the communication bridge between the client application and the backend services. It is implemented using FastAPI and is responsible for handling

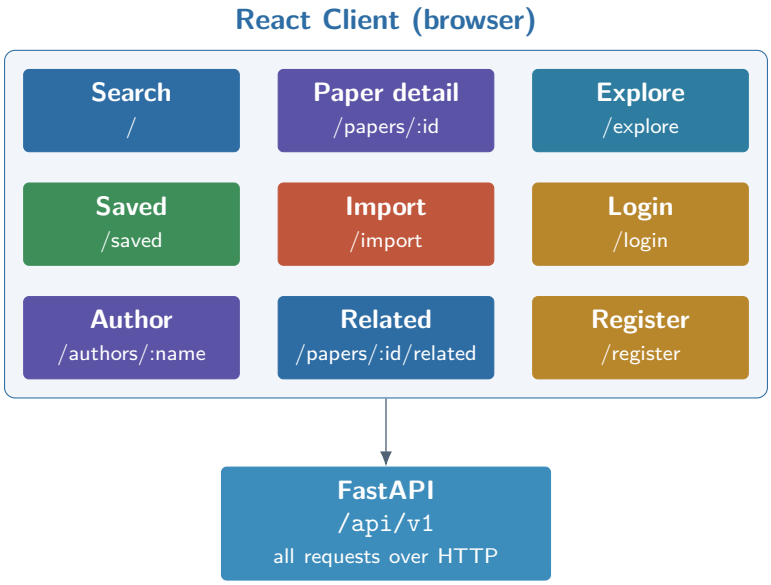


Figure 2: Client layer and web pages.

HTTP requests, validating input data, managing authentication and authorization, and forwarding requests to the service layer.

All API endpoints are available under the common prefix `/api/v1`. The API is stateless, meaning that user identity is provided through JWT Bearer tokens for protected requests, and no server-side session information is stored. Public endpoints provide access to features such as searching, paper retrieval, exploration, registration, and login. Protected endpoints require authentication, while administrative operations such as importing, deleting papers, and monitoring tasks require an additional `admin` role.

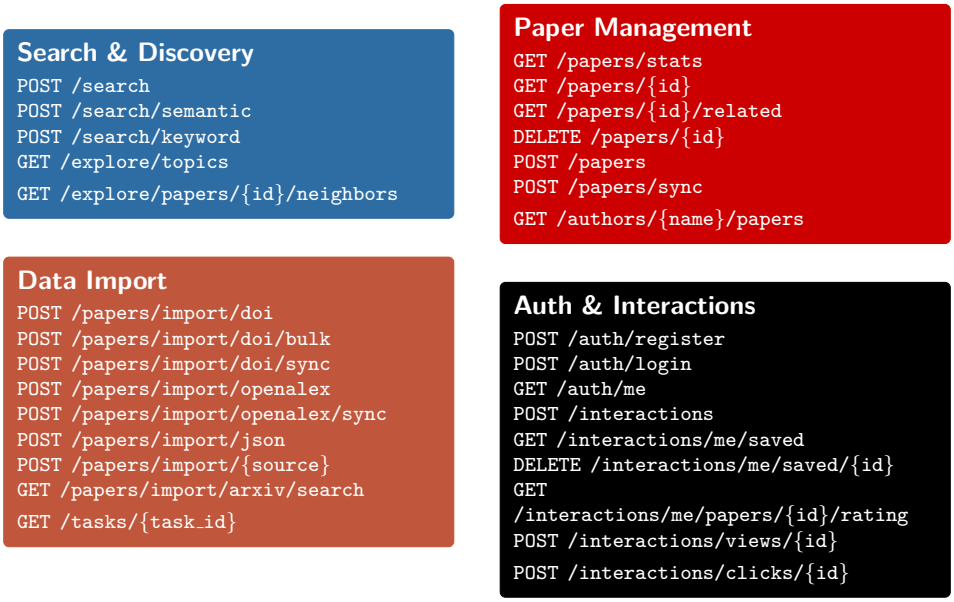


Figure 3: REST API endpoints of the Citexis backend.

Figure 3 summarizes the REST API endpoints of Citexis, grouped into search and discovery, paper management, data import, and authentication and interaction functionalities. The API layer validates incoming requests and forwards them to the corresponding

services, while long-running operations such as bulk imports are handled asynchronously through the background processing layer.

2.5 Service Layer

The service layer implements the core business logic of Citexis, including literature discovery (hybrid, keyword, and semantic search), recommendation, exploration, ingestion, indexing, and authentication. It acts as the coordination layer between the API layer and the underlying data services. It integrates MongoDB for document storage, Elasticsearch for BM25 keyword retrieval, and Pinecone for vector-based semantic search. Shared service instances are initialized when the FastAPI application starts and are provided to route handlers using dependency injection. Table 4 provides an overview of the main service components and their responsibilities, which are described in the following sections.

Component	Responsibility
Hybrid Search	Combines results of both semantic and keyword retrieval using Reciprocal Rank Fusion (RRF) and retrieves complete metadata from MongoDB
Keyword Search	Performs keyword-based retrieval using BM25 through Elasticsearch `multi_match` queries
Semantic Search	Performs semantic similarity-based vector retrieval using Pinecone with re-ranking algorithm
Recommendation	Generates recommendations by combining content, citation, and collaborative filtering through weighted RRF fusion
Explore	Supports research discovery through clustering papers using tags, citation relationships, and author-based exploration
Ingestion	Collects, normalizes, stores, and indexes research paper details and metadata
Indexing	Manages insertion, updates, and deletion of Elasticsearch documents and Pinecone vectors
Authentication	Handles user registration, login, JWT token generation, and token validation

Table 4: Service layer components and their responsibilities.

2.6 Data Layer

The data layer uses MongoDB to store metadata of each paper, user accounts, and interaction data. Elasticsearch and Pinecone provide specialized retrieval capabilities and are updated asynchronously from MongoDB by background workers. The synchronization status of each paper is tracked using the `embedding_status` field, which can have the values `pending`, `processing`, `indexed`, or `failed`. Redis is used only by Celery as the message broker and result backend for asynchronous tasks and does not store application data.

MongoDB contains three main collections for storing papers, users, and user interactions, whose schemas are presented in Tables 5–7. A unique index on the `doi` field prevents duplicate papers, while a unique index on `email` ensures that each user account is registered only once. The shared enumeration values used across the database are listed in Table 8.

Elasticsearch maintains a separate search index for all papers. To improve keyword search, the paper title and abstract are indexed using a custom text analyzer with lowercase conversion, English stop-word removal, and stemming. The BM25 ranking algorithm retrieves papers by giving higher priority to matches in the title than in the abstract.

Additional fields, such as authors, publication year, venue, tags, and citation count, support filtering and ranking. Each Elasticsearch document uses the same identifier as its corresponding MongoDB paper.

Pinecone stores vector embeddings of research papers to enable semantic similarity search. Each record is identified by the MongoDB paper ID. Paper title, abstract, and optional tags are combined to generate vector embeddings. Additional metadata, including the publication year, venue, citation count, and tags, is stored alongside each embedding to support filtering during retrieval. A re-ranking model is applied at query time to improve the quality of the returned results.

Field	Type	Description
`_id`	ObjectId	Primary key
`doi`	string (optional)	Normalised DOI
`title`	string	Paper title
`abstract`	string	Abstract text
`authors`	array of object	Author Detail
`year`	integer (optional)	Publication year
`venue`	string (optional)	Journal or conference
`tags`	array of string	Topic keywords
`citations`	array of string	Reference DOIs
`cited_by_count`	integer	External citation count
`full_text_url`	string (optional)	Link to full text
`embedding_status`	enum	Indexing state
`pinecone_id`	string (optional)	Pinecone record id when indexed
`es_id`	string (optional)	Elasticsearch document id when indexed

Table 5: MongoDB `papers` collection schema.

Field	Type	Description
`_id`	ObjectId	Primary key
`email`	string	Unique, lowercased account email
`password_hash`	string	Hashed password
`role`	enum	`user` or `admin`
`is_active`	boolean	Account enabled flag
`created_at`	datetime	Account creation timestamp (UTC)

Table 6: MongoDB `users` collection schema.

Field	Type	Description
`_id`	ObjectId	Primary key
`user_id`	string	Referenced user id
`paper_id`	string	Referenced paper id
`interaction_type`	enum	Types of interaction
`rating`	float (optional)	Score in $[0, 5]$ for rating interactions

Table 7: MongoDB `user_interactions` collection schema.

Enumeration	Values
EmbeddingStatus	pending, processing, indexed, failed
InteractionType	view, save, click, rating
UserRole	user, admin

Table 8: Shared enumerations used across MongoDB collections.

2.7 Background Processing

The background processing layer handles long-running ingestion and indexing tasks outside the FastAPI request–response cycle, keeping API handlers responsive during bulk imports and external metadata retrieval. Asynchronous jobs are implemented using Celery with Redis as the message broker and result backend. Workers serialize payloads as JSON, acknowledge tasks late (`task_acks_late`), and prefetch a single job at a time (`worker_prefetch_multiplier = 1`) to avoid monopolizing external APIs and database connections. On startup, each worker ensures that the required MongoDB indexes are available.

Import and indexing operations that exceed interactive latency return HTTP `202 Accepted` with a Celery `task_id`. Administrators can monitor progress through `GET /tasks/{task_id}`, which provides task status, results, or error information. Failed tasks are retried up to three times with bounded back-off (60 s for single-paper jobs and 120 s for bulk jobs). Each worker creates temporary MongoDB, Elasticsearch, and Pinecone clients for task execution and closes them after completion.

All asynchronous jobs invoke the shared ingestion pipeline, including metadata retrieval, optional CORE enrichment, normalization, MongoDB persistence, and secondary indexing. Duplication of papers is prevented by normalizing the DOI and using a unique index on `papers.doi`. As a result, importing the same DOI again does not create another document. The registered Celery tasks are summarized in Table 9.

Task	Responsibility
ingest_doi_task	Resolve a single DOI via Crossref and ingest one paper
ingest_dois_bulk_task	Resolve many DOIs via Crossref and ingest the resulting set
ingest_openalex_task	Search OpenAlex and ingest matching papers
ingest_source_task	Generic source ingestion for OpenAlex or Crossref
ingest_json_bulk_task	Save and index a validated batch of papers
index_paper_task	Import a single paper provided by the user

Table 9: Celery background tasks for paper ingestion and indexing.

Synchronous jobs remain available for small interactive operations, while bulk DOI imports, OpenAlex imports, JSON uploads, and queued indexing tasks use the background processing workflow to avoid blocking API workers.

3 Implementation

This section describes the implementation of the main discovery and security features of the service layer.

3.1 Ingestion Pipeline

The ingestion pipeline is responsible for collecting, normalizing, storing, and indexing research paper metadata from multiple external sources, as shown in Figure 4. Papers can enter the pipeline in four ways: DOI or topic-based lookup via the OpenAlex and Crossref APIs, arXiv search, direct JSON file upload, or manual entry through the admin interface.

For DOI-based imports, the pipeline retrieves paper metadata from OpenAlex or Crossref. An optional step uses the CORE API to obtain full-text PDF links of the papers when available. The collected metadata, including title, abstract, authors, year, venue, tags, and citations, is normalized into the MongoDB paper schema.

Before storing a paper, the pipeline checks the normalized DOI against the unique DOI index to prevent duplicates. If the paper already exists, the import is skipped, allowing repeated imports to run safely without creating duplicate records. Otherwise, the paper metadata is stored in MongoDB as the primary record.

After saving the paper, the combined text (title, abstract, and tags) is converted into an embedding and stored in Pinecone for semantic search. The same document is indexed in Elasticsearch for BM25 keyword search. Successful indexing updates the `embedding_status` field to `indexed`, while failed synchronization can be retried by background tasks.

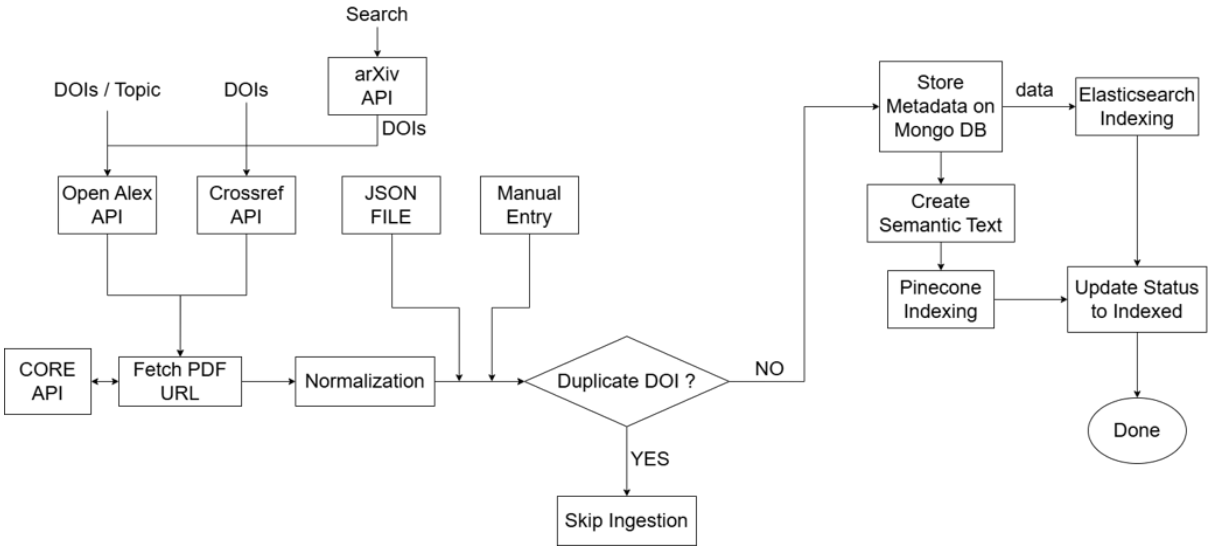


Figure 4: Ingestion Pipeline for Papers

3.2 Hybrid Search

Hybrid search combines semantic and keyword retrieval results, as shown in Figure 5. When a user submits a query, it is processed by two parallel pipelines: semantic search and keyword search. Each pipeline returns a ranked list of candidate papers, which are combined using Reciprocal Rank Fusion (RRF). The final ranked paper IDs are then used to retrieve complete metadata from MongoDB before returning the response to the user.

The semantic search pipeline in Figure 6 converts the query into an embedding vector using the embedding model. Pinecone compares this vector with stored paper embeddings using cosine similarity and returns the most similar candidates. A re-ranking step

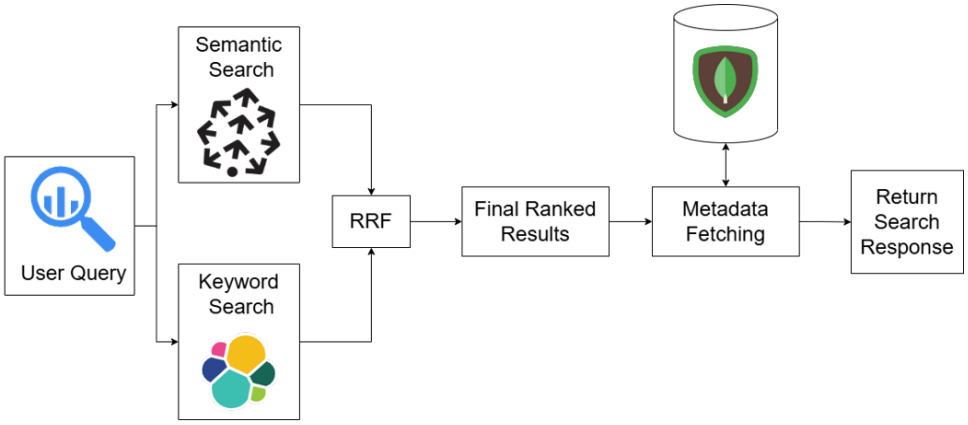


Figure 5: Hybrid Search Mechanism

improves the ranking of these candidates before returning the final results. This approach can retrieve papers with similar concepts and context.

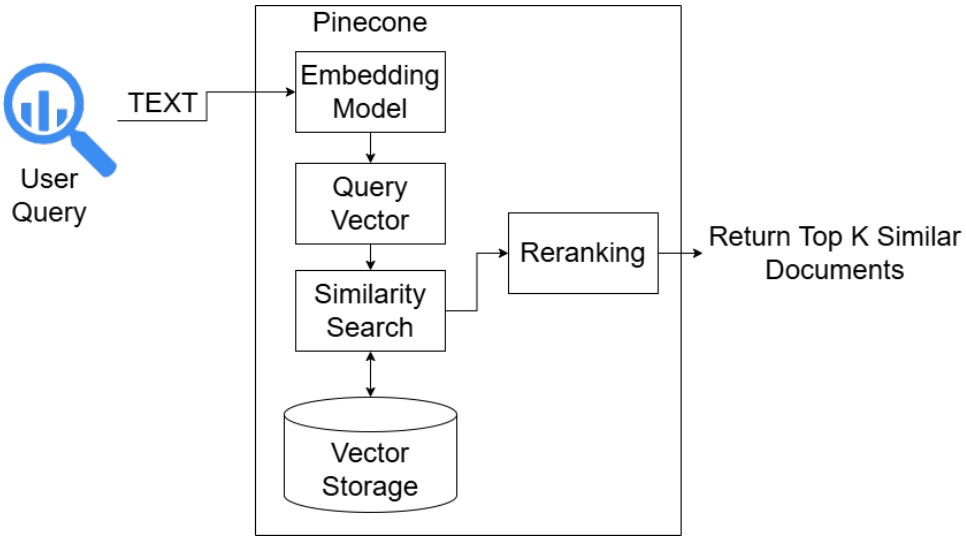


Figure 6: Semantic Search Mechanism

The keyword search pipeline in Figure 7 processes the query using Elasticsearch. The query terms are matched against an inverted index, and documents are ranked using BM25. This approach is effective for retrieving papers containing exact or closely related terms.

Since semantic search and keyword search produce scores on different scales (cosine similarity and BM25), their raw scores cannot be directly combined. Citexis uses Reciprocal Rank Fusion (RRF), a training-free method that merges heterogeneous ranked lists based on document positions rather than score values [1]. For document d, the fused score is:

$$\mathrm{RRF}(d) = \sum_{r \in \text{rankers}} \frac{1}{k + \mathrm{rank}_r(d)}$$

where $\mathrm{rank}_r(d)$ is the rank of document d in ranker r, and k is a smoothing constant set to 60. Rank-based fusion avoids score normalization and allows effective combination

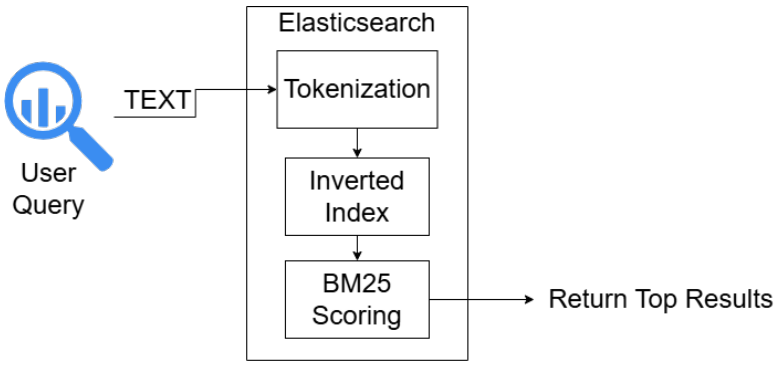


Figure 7: Keyword Search Mechanism

of different retrieval methods without requiring labeled training data. A worked example is shown in Figure 8.

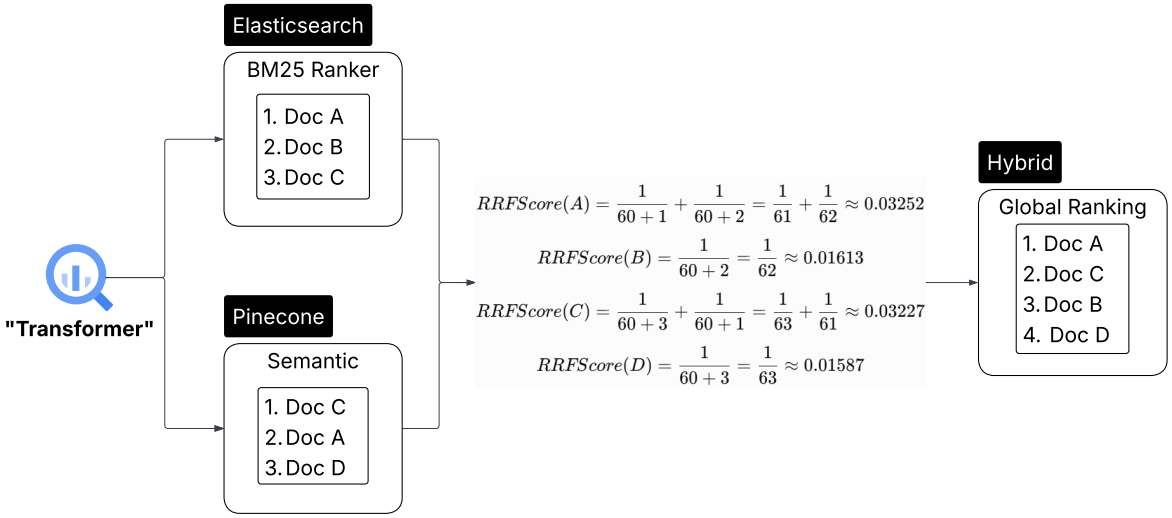


Figure 8: Reciprocal Rank Fusion

3.3 Recommendation Engine

The recommendation engine generates related-paper suggestions by combining content-based, citation-based, and collaborative filtering strategies through weighted Reciprocal Rank Fusion (RRF), as shown in Figure 9. The three recommendation lists are combined using weighted RRF, with content-based recommendations receiving the highest weight ($W_1 = 0.5$), followed by citation-based ($W_2 = 0.3$) and collaborative filtering ($W_3 = 0.2$). This weighting reduces the impact of the cold-start problem, as content and citation metrics do not depend on user interaction history. The final ranked paper IDs are used to retrieve complete metadata from MongoDB and returned to the client.

The content-based approach in Figure 10 recommends papers based on semantic similarity. Given a paper ID, the system retrieves its title, abstract, and tags to generate an embedding and searches Pinecone for the most similar papers using cosine similarity.

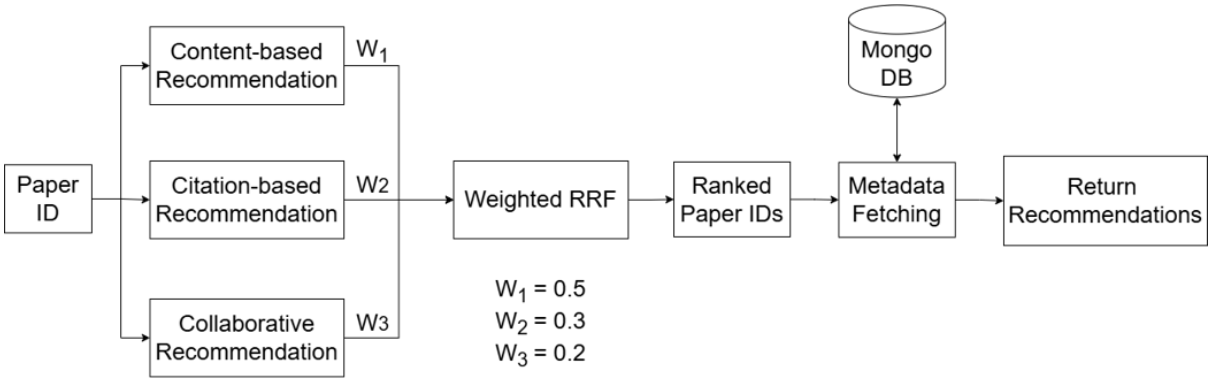


Figure 9: Recommendation Engine

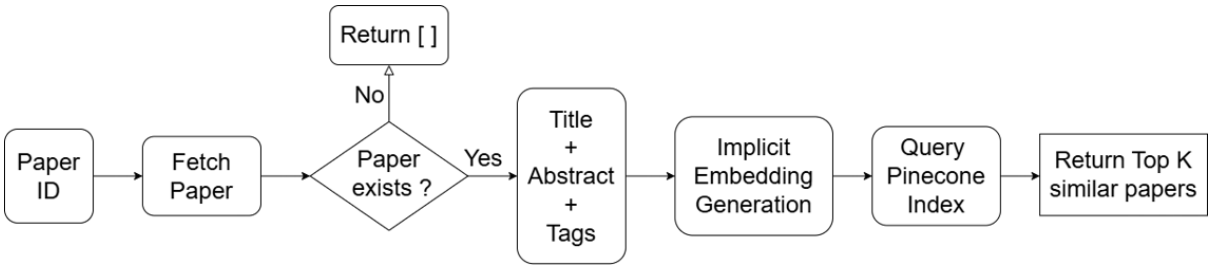


Figure 10: Content-based

The citation-based approach in Figure 11 uses shared references between papers. The system compares the citations of the selected paper with other papers and ranks candidates based on the number of shared citations. Papers with higher citation overlap are selected as recommendations.

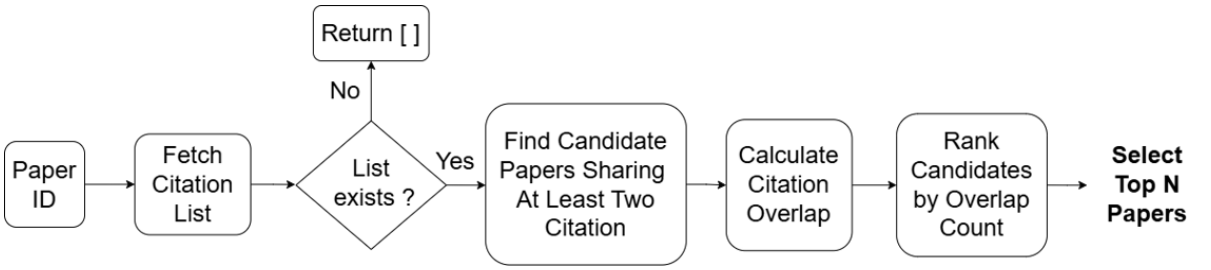


Figure 11: Citation-based

The collaborative filtering approach in Figure 12 uses user interaction data, including views, saves, ratings, and clicks. It identifies users who interacted with the selected paper, finds other papers they interacted with, and ranks candidates based on shared user interactions.

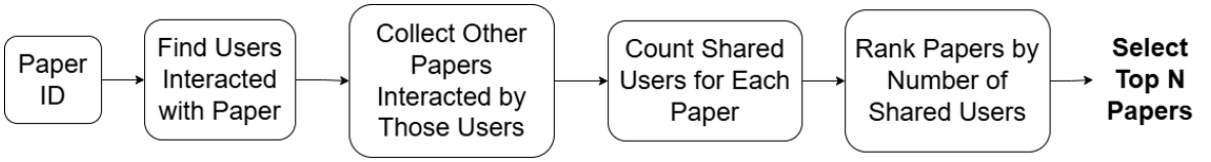


Figure 12: Collaborative-based

3.4 Authentication and Access Control

Citexis implements JWT-based authentication to support user-specific functionality and administrator-controlled data management. User passwords are securely stored using bcrypt hashing, while access tokens are generated and signed using the HS256 algorithm. An initial administrator account is provisioned from environment variables during application startup, and role-based access control is enforced at the route level through FastAPI dependency injection.

Role	Capabilities
Public	Search literature, read papers, explore topics, and browse authors
Authenticated user	Save papers, rate papers, and record interaction history such as views and clicks
Administrator	Import and remove papers, and monitor Celery task execution status

Table 10: User roles and associated capabilities.

4 Experimental Setup

The experimental setup consists of three main components: the embedding model used for semantic retrieval, the re-ranking model used to improve results, and the hybrid fusion strategy used to combine multiple search methods.

4.1 Embedding Model

The semantic retrieval component uses `llama-text-embed-v2` through Pinecone to generate dense vector representations of research papers.

Parameter	Value
Model	`llama-text-embed-v2`
Base model	Llama 3.2 (1B parameters)
Dimension	1024
Similarity metric	Cosine similarity

Table 11: Embedding model configuration.

4.2 Ranking Model

A re-ranking step improves semantic search results using the multilingual `bge-reranker-v2-m3` model.

Parameter	Value
Model	`bge-reranker-v2-m3`
Base model	BGE-M3 (560M parameters)
Configuration	`PINECONE_RERANK_ENABLED=true`

Table 12: Reranking model configuration.

4.3 Hybrid Fusion Parameters

Keyword and semantic search results are combined using Reciprocal Rank Fusion (RRF). The main fusion parameters are listed below.

Parameter	Default	Description
SEARCH_TOP_K_PER_SOURCE	50	Number of candidates retrieved from each search method
SEARCH_RRF_K	60	Constant used to balance ranking scores during RRF fusion
SEARCH_FUSION_TOP_N	20	Number of candidates kept after combining search results
SEARCH_FINAL_TOP_N	10	Number of final results shown to users

Table 13: Hybrid search fusion parameters.

5 Discussion

5.1 Technical Challenges and Solutions

- **Multi-index consistency:** Maintaining consistency between MongoDB, Elasticsearch, and Pinecone is challenging because failures can interrupt synchronization. This is addressed using `embedding_status` tracking, automatic Elasticsearch index initialization, and maintenance scripts for re-indexing and cleanup.
- **Collaborative filtering cold start:** New users and papers have limited interaction data, which reduces the effectiveness of collaborative filtering. Therefore, content-based and citation-based recommendations receive higher weights, while collaborative filtering contributes less until sufficient interaction data becomes available.
- **Search score combination:** BM25 and vector similarity scores are generated on different scales and cannot be directly combined. Citexis addresses this issue using Reciprocal Rank Fusion (RRF) with $k = 60$, which combines results based on ranking positions.
- **External API dependency:** External API-based sources such as OpenAlex and Crossref may introduce delays or failures. Asynchronous Celery tasks with retries and failure tracking improve the reliability of the import process.

5.2 Evaluation and Reflection

The system was tested with more than 600 papers collected from OpenAlex and Crossref. The evaluation focused on whether the main features worked correctly and responded quickly.

- **Performance:** Interactive search runs keyword and semantic searches in parallel, retrieving up to 50 candidates from each method. The results are combined using Reciprocal Rank Fusion, and only the final top results are loaded from MongoDB to keep search responses fast. Optional reranking is applied only to the selected top candidates to reduce the computational cost. Time-consuming tasks such as bulk ingestion and indexing are moved off the request path to Celery workers, so they does not block interactive search.

- **Scalability:** The polyglot architecture allows each storage system to scale independently. MongoDB, Elasticsearch, and Pinecone can be expanded or distributed separately based on their requirements, while the stateless FastAPI backend with JWT authentication can be replicated across multiple instances. Ingestion performance can be improved by adding more Celery workers. Since MongoDB stores the primary data, Elasticsearch and Pinecone indexes can be rebuilt or adjusted as the paper collection grows. The current prototype runs on a single-node Docker Compose setup, while a production deployment would require container orchestration and appropriate cluster configuration.

- **Trade-offs:** Polyglot persistence improves retrieval performance by using different storage systems for different tasks, but it also introduces additional consistency challenges between storage systems. These challenges are managed through embedding status tracking and re-indexing scripts. Reciprocal Rank Fusion was selected because it provides reliable results without requiring training. Hosted embedding models and managed services simplify development and reduce maintenance effort, but they introduce external dependencies, additional costs, and limitations due to the lack of domain-specific fine-tuning. The recommendation weights are fixed and easy to understand, prioritizing simplicity and interpretability over maximum accuracy.

5.3 Limitations

Limitation	Description
External data sources	Importing papers depends on external APIs, which may introduce rate limits, delays, or availability issues.
API security	Role-based access control is implemented, but additional mechanisms such as rate limiting and OAuth2/SSO are not included.
Embedding dependency	Semantic search uses a hosted pretrained embedding model without additional fine-tuning for the research domain.

Table 14: System limitations.

6 Conclusion and Future Work

Citexis provides a research paper discovery platform that combines hybrid search and recommendation approaches to improve literature exploration. The system integrates semantic retrieval, keyword-based retrieval, and recommendation strategies based on content, citation relationships, and user interactions. Through this combination, Citexis addresses the limitations of traditional keyword-only search and supports more effective discovery of scientific literature.

The implemented platform provides the core functionalities required for a scalable research discovery system, including paper ingestion, intelligent search, recommendation, user personalization, and administrator-controlled data management. The modular architecture enables independent development and maintenance of the individual components while supporting reproducible deployment. The future work proposed to further improve the system can be seen below:

Area	Description
Personalized recommendations	Improved user modeling using richer interaction data and advanced recommendation techniques
Custom embedding model	Domain-specific fine-tuning of embedding models for scientific literature retrieval
Scheduled ingestion	Automatic periodic updates of the research data from external sources
API security	Additional security mechanisms including rate limiting and OAuth2/SSO support

Table 15: Proposed future enhancements.

References

[1] Gordon V. Cormack, Charles L A Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '09, page 758–759, New York, NY, USA, 2009. Association for Computing Machinery.

[2] Crossref. Crossref. `https://www.crossref.org/`, 2026. Accessed: 2026-07-17.

[3] Elastic. Elasticsearch documentation. `https://www.elastic.co/elasticsearch`, 2026. Accessed: 2026-07-17.

[4] FastAPI Team. Fastapi documentation. `https://fastapi.tiangolo.com/`, 2026. Accessed: 2026-07-17.

[5] MongoDB Inc. Mongodb documentation. `https://www.mongodb.com/docs/`, 2026. Accessed: 2026-07-17.

[6] Pinecone Systems Inc. Pinecone documentation. `https://docs.pinecone.io/guides/get-started/overview`, 2026. Accessed: 2026-07-17.

[7] Jason Priem, Heather Piwowar, and Richard Orr. Openalex: A fully-open index of scholarly works, authors, venues, institutions, and concepts, 2022.

[8] React Team. React documentation. `https://react.dev/learn`, 2026. Accessed: 2026-07-17.

[9] Francesco Ricci, Lior Rokach, and Bracha Shapira. *Recommender Systems Handbook*. Springer, 2nd edition, 2015.

[10] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Found. Trends Inf. Retr.*, 3(4):333–389, April 2009.

A Setup and Installation Guide

This appendix describes how to deploy Citexis for local development and demonstration. The application is fully containerized using Docker Compose, which manages the Elasticsearch, Redis, FastAPI backend, Celery worker, React frontend, and Elasticsearch initialization service. MongoDB and Pinecone are external services and must be configured before starting the application. Application services are not run directly on the host; all runtime components are started through Docker Compose.

A.1 Prerequisites

Before running the system, ensure that the following requirements are met:

- Docker and Docker Compose are installed.
- A MongoDB deployment is available (MongoDB Atlas is recommended).
- A Pinecone account is configured with an index that supports server-side text embedding (e.g., `llama-text-embed-v2`) and the `papers-abstract` namespace.
- (Optional) A CORE API key is available if full-text URL enrichment via CORE is required during DOI and OpenAlex ingestion.

On Linux, Docker Compose uses `network_mode: host` so containers can reach external MongoDB and Pinecone endpoints and bind application ports directly on the host.

A.2 Environment Configuration

Application settings are stored in an `.env` file shared by the API, Celery worker, and Elasticsearch initialization service.

Important

For evaluation and demonstration purposes, the submitted project already contains a configured `.env` file. Therefore, the application can be started directly without additional environment configuration. The values shown below are examples of the required configuration parameters; sensitive credentials are omitted for security reasons.

For a custom deployment, create the environment file from the provided template:

```
cp .env.example .env
openssl rand -hex 32 # generate JWT_SECRET_KEY
```

The following environment variables are required:

```
# Application
APP_ENV=development
DEBUG=true

# MongoDB
MONGODB_URI=<mongodb-atlas-connection-string>
MONGODB_DB=citexis
```

```
# Redis
REDIS_HOST_PORT=6380
REDIS_URL=redis://localhost:6379/0

# Elasticsearch
ELASTICSEARCH_URL=http://localhost:9200
ELASTICSEARCH_INDEX=papers

# Pinecone
PINECONE_API_KEY=<pinecone-api-key>
PINECONE_INDEX=citexis
PINECONE_NAMESPACE=papers-abstract
PINECONE_TEXT_FIELD=text
PINECONE_RERANK_ENABLED=true
PINECONE_RERANK_MODEL=bge-reranker-v2-m3

# Ingestion
CORE_API_KEY=<core-api-key>
INGESTION_ENRICH_CORE=true
INGESTION_MAX_CITATIONS=1000

# Bulk JSON import
JSON_IMPORT_MAX_BYTES=1048576
JSON_IMPORT_MAX_PAPERS=500

# Search configuration
SEARCH_TOP_K_PER_SOURCE=50
SEARCH_RRF_K=60
SEARCH_FUSION_TOP_N=20
SEARCH_FINAL_TOP_N=10

# Authentication (JWT)
JWT_SECRET_KEY=<random-secret-key>
JWT_ALGORITHM=HS256
JWT_ACCESS_TOKEN_EXPIRE_MINUTES=1440

# Bootstrap administrator account
AUTH_ADMIN_EMAIL=<administrator-email>
AUTH_ADMIN_PASSWORD=<administrator-password>
```

Docker Compose automatically configures internal service connections for containerized components. In particular, `ELASTICSEARCH_URL` and `REDIS_URL` are overridden for the `api`, `worker`, and `es-init` services when running with host networking. MongoDB and Pinecone connection parameters are loaded directly from the environment file.

A.3 Starting the Application

From the project root directory, stop any existing Elasticsearch container that may already occupy port 9200, then build and start the stack:

```
docker stop elasticsearch 2>/dev/null || true
docker compose up --build
```

Docker Compose launches the services listed in Table 16. During startup, the `es-init` service creates the Elasticsearch index if necessary and backfills it from MongoDB when

the index is new or contains fewer documents than the database. The API and Celery worker start only after this initialisation has completed successfully.

Service	Port	Purpose
`elasticsearch`	9200	BM25 keyword search engine
`redis`	6380*	Celery message broker and result backend
`es-init`	—	One-shot Elasticsearch index creation and MongoDB backfill
`api`	8000	FastAPI backend server
`worker`	—	Background bulk import processing (Celery)
`frontend`	5173	React web application (Vite; proxies `/api` to the API)

Table 16: Docker Compose services used by Citexis. *Default host port; configurable via `REDIS_HOST_PORT`.

A.4 Verifying the Deployment

After all services have started successfully, the following endpoints can be used to confirm that the deployment is operational:

- Web application: `http://localhost:5173`
- Interactive API documentation (Swagger UI): `http://localhost:8000/docs`
- Elasticsearch cluster health: `http://localhost:9200/_cluster/health`

Redis is available on host port `6380` by default (configurable via `REDIS_HOST_PORT`) and is used internally by Celery; it is not accessed through a browser.

Sign in with the administrator credentials defined in the provided `.env` file to access administrative features such as bulk paper import and paper deletion. Bulk imports (multi-DOI via Crossref, OpenAlex search, and JSON upload) are processed asynchronously by Celery. Each submitted job returns a `task_id`, and its progress can be monitored using `GET /api/v1/tasks/{task_id}` until completion.

A sample dataset for testing is included in `fixtures/sample_papers.json` and can be imported through the web interface.

A.5 Index Maintenance

If the Elasticsearch or Pinecone indexes become inconsistent with MongoDB, they can be reconciled using the provided maintenance scripts:

```
docker compose run --rm api \
    python -m scripts.reindex_elasticsearch --apply

docker compose run --rm api \
    python -m scripts.reindex_pinecone --apply

docker compose run --rm api \
    python -m scripts.cleanup_orphan_indexes --apply
```

B User Interface Walkthrough

This appendix presents the main pages of the Citexis user interface. The figures illustrate the primary workflows available in the prototype, including user authentication, paper discovery, recommendation, exploration, personalization, and administrator operations.

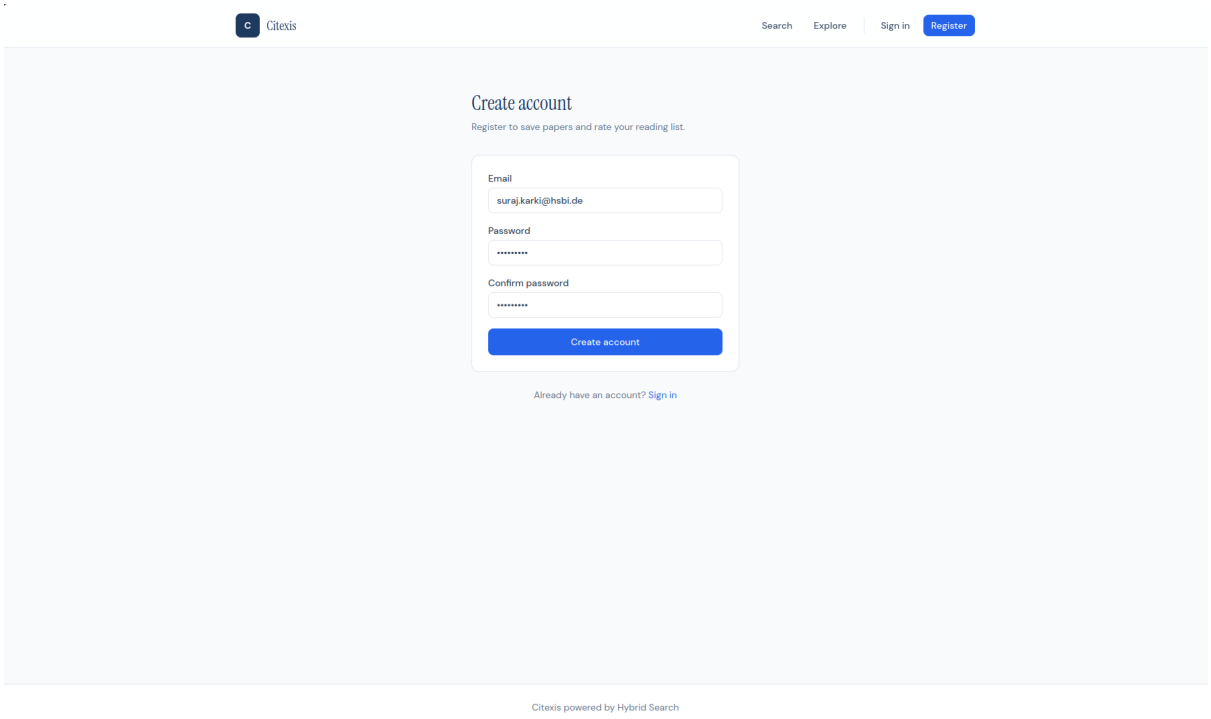


Figure 13: User registration interface of Citexis.

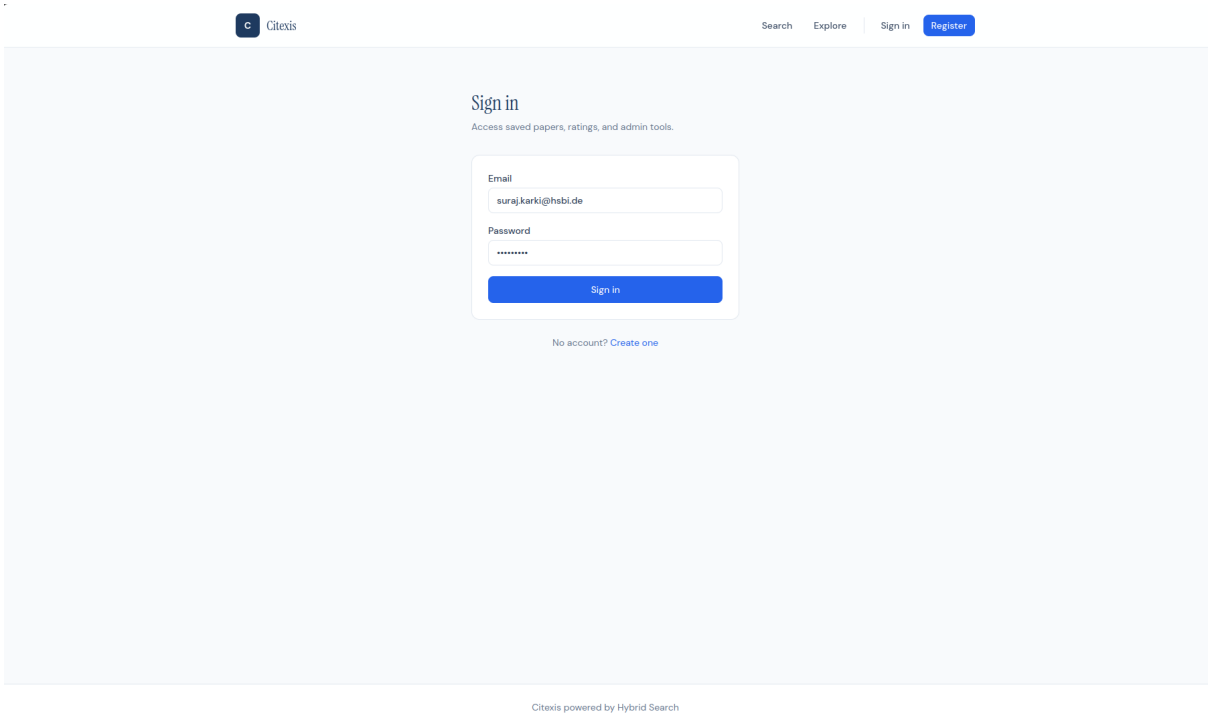


Figure 14: User login interface of Citexis.

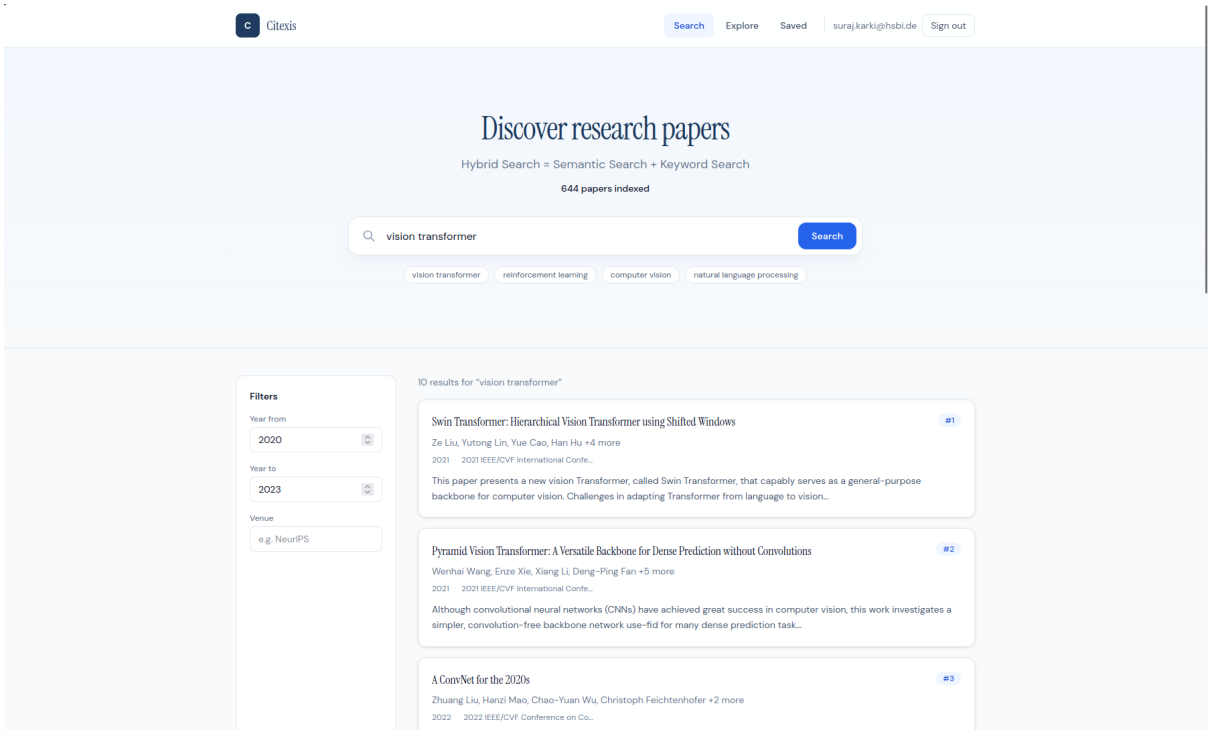


Figure 15: Hybrid search interface combining keyword and semantic paper retrieval.

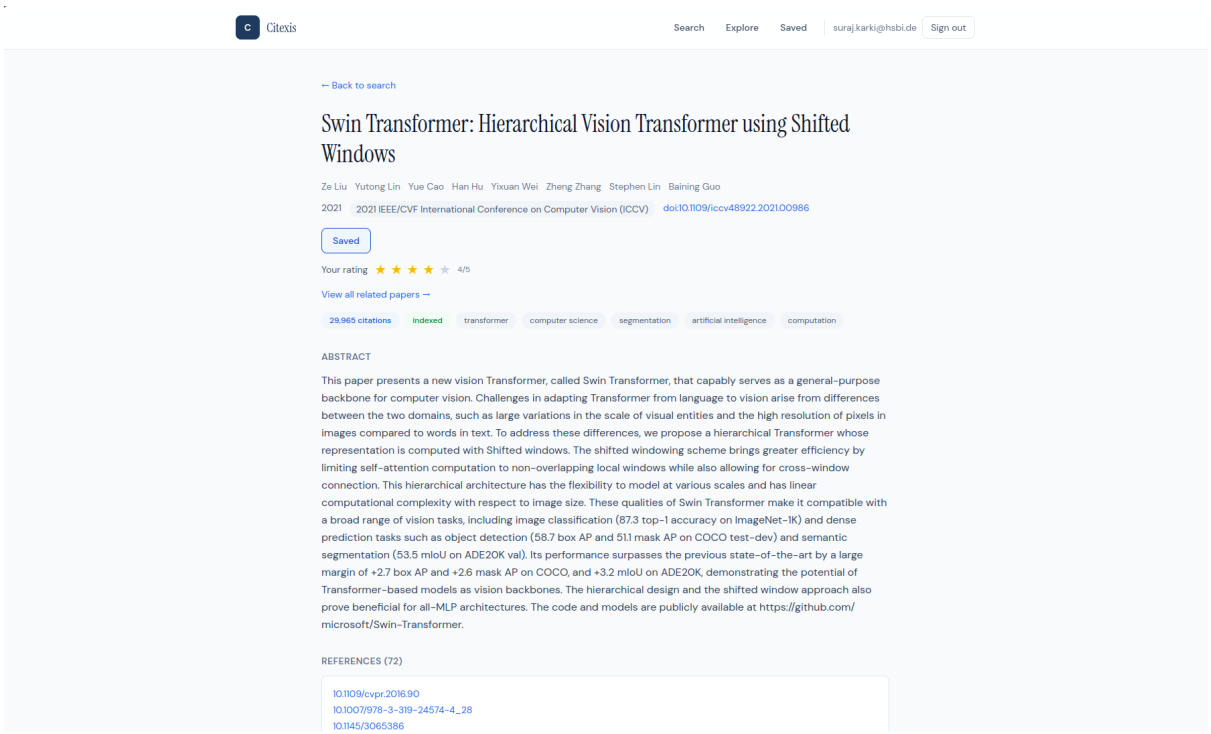


Figure 16: Detailed paper view displaying metadata, abstract, citations, and related information.

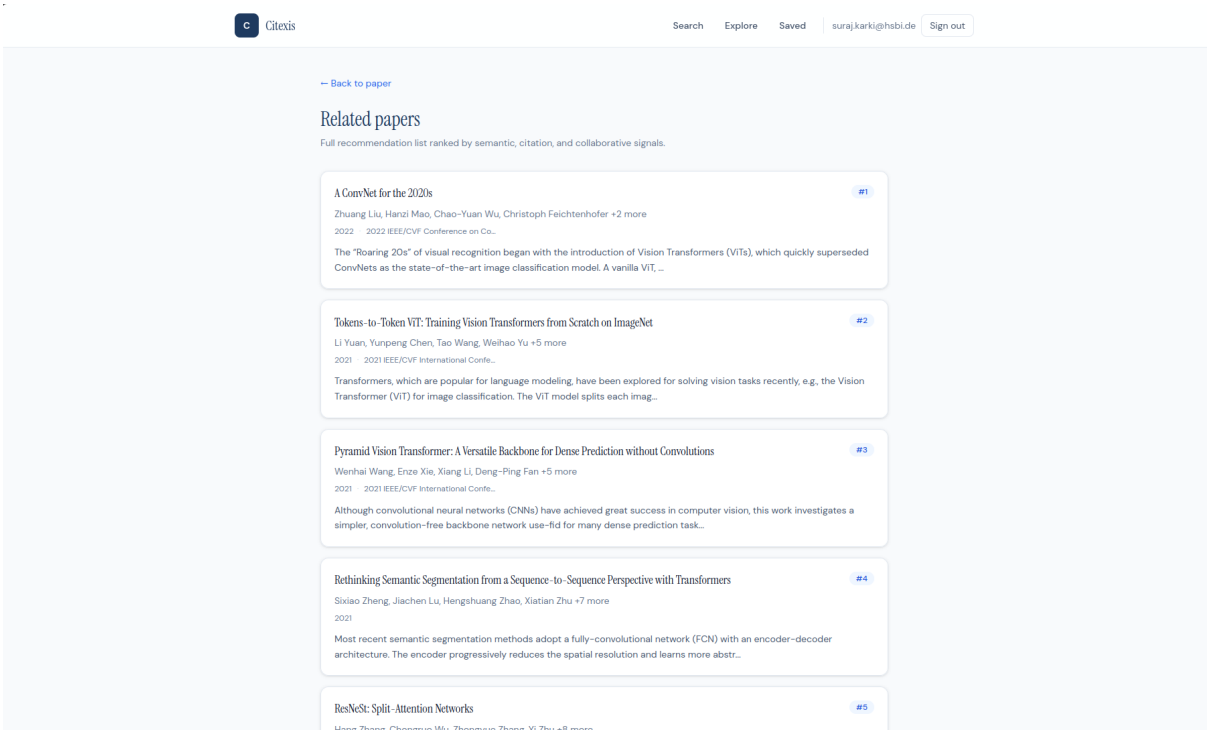


Figure 17: Related paper recommendation interface.

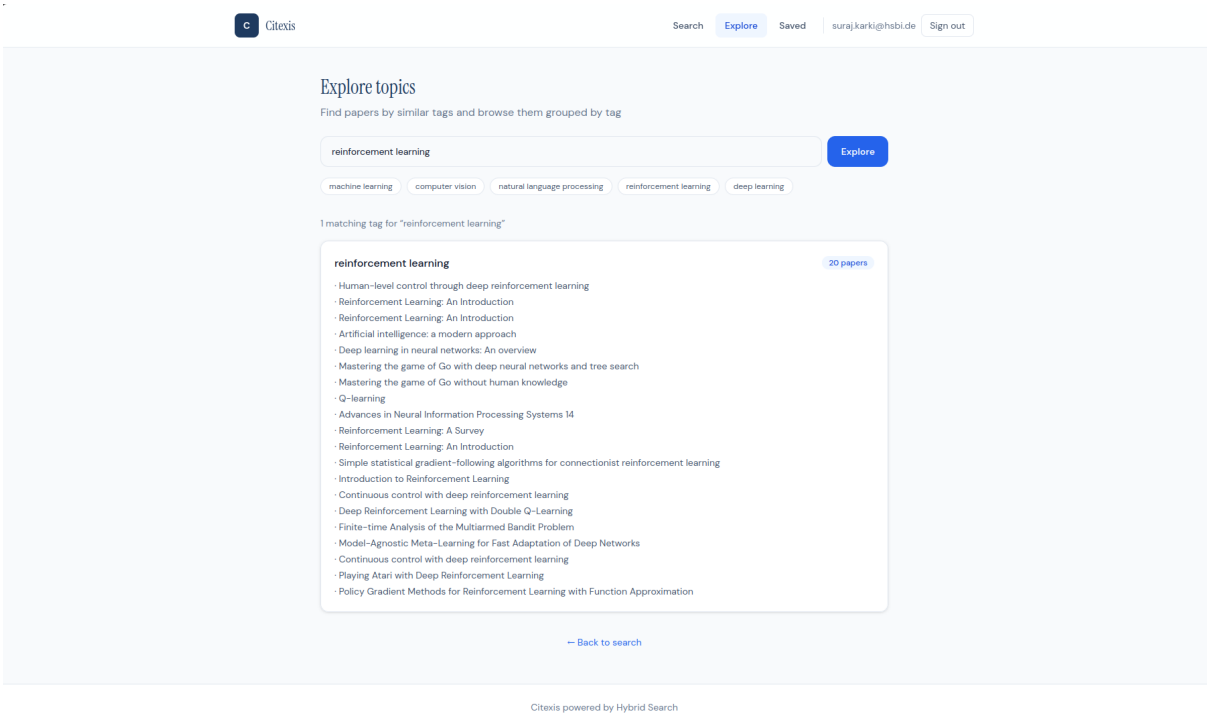


Figure 18: Topic exploration interface for discovering research areas and papers.

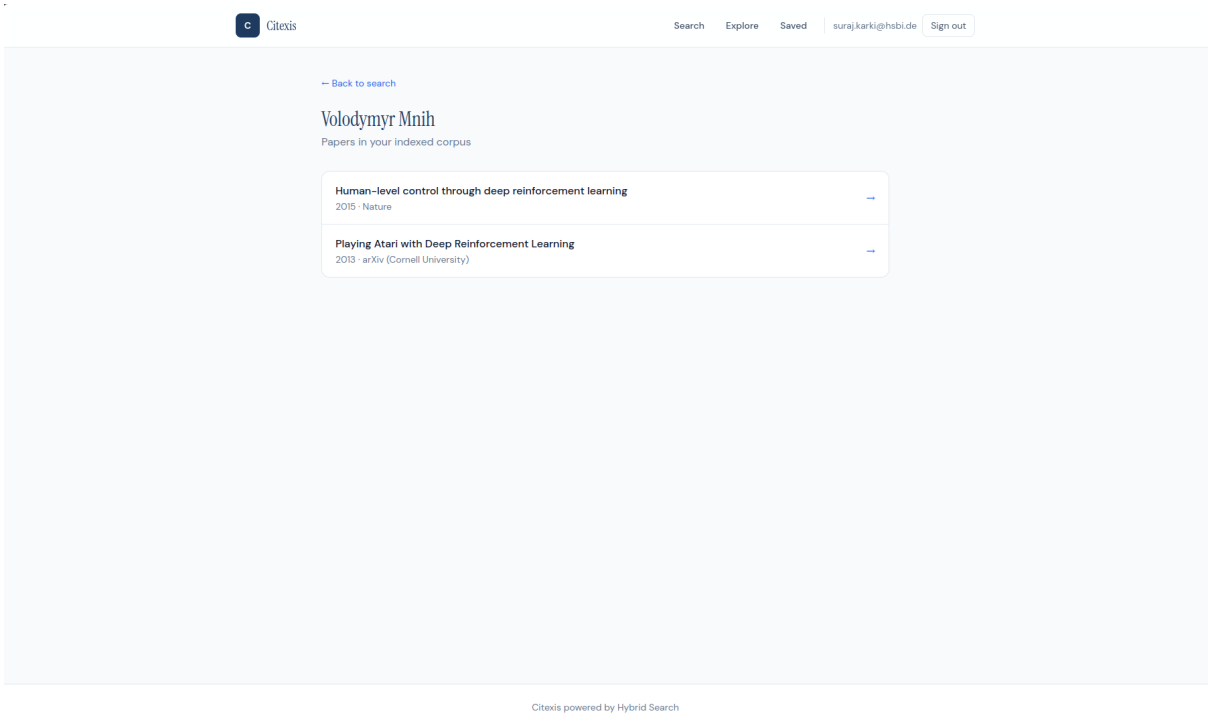


Figure 19: Author browsing interface showing papers associated with researchers.

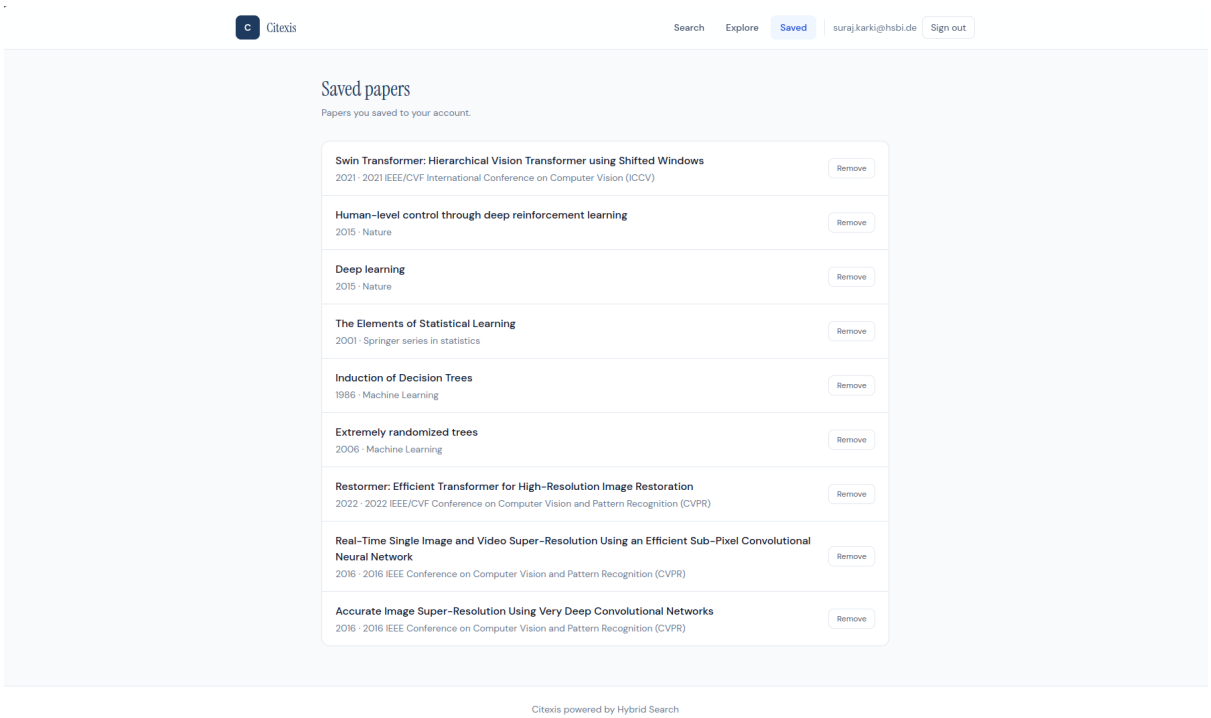


Figure 20: Saved papers interface for managing personalized paper collections.

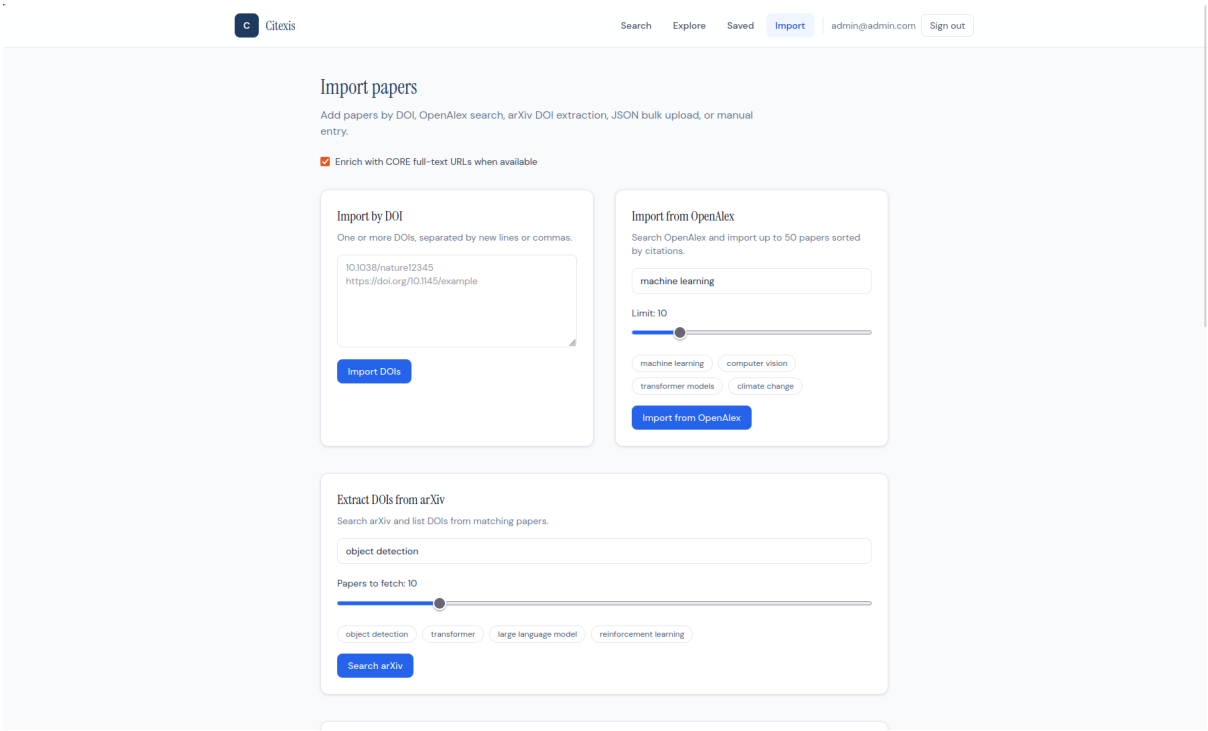


Figure 21: Administrator interface for initiating paper ingestion and bulk import operations.

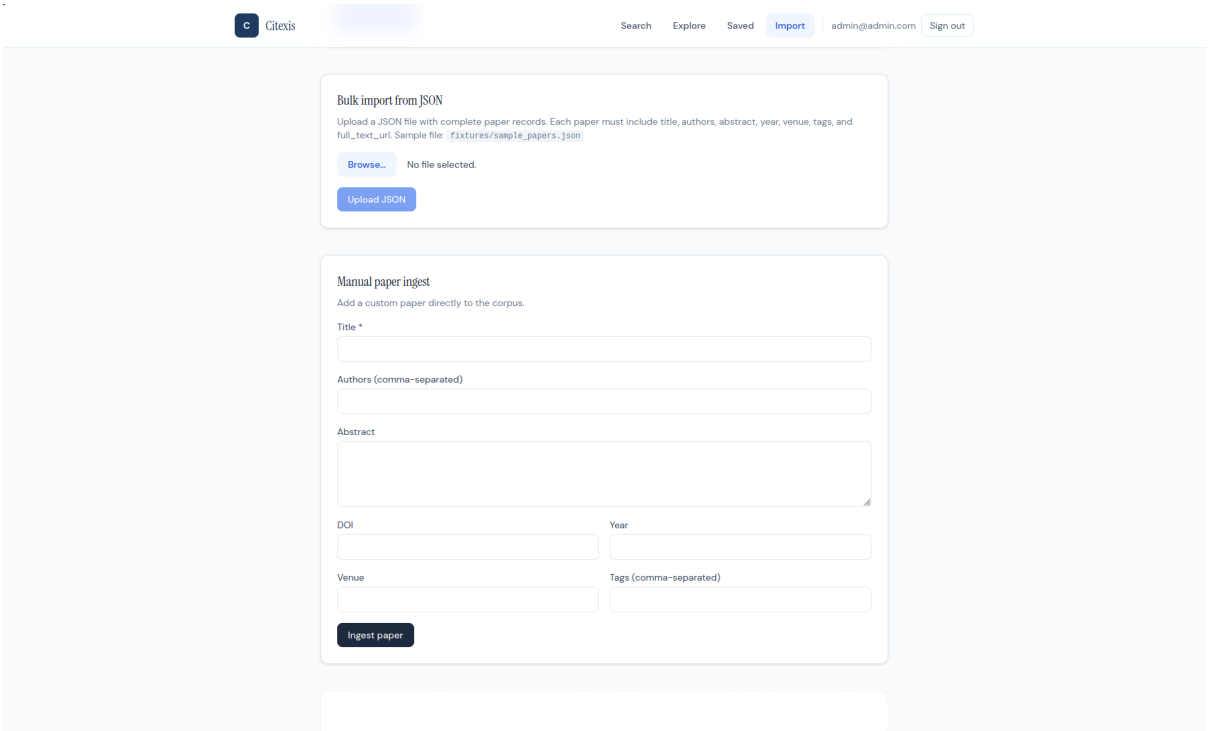


Figure 22: Administrator interface showing import progress and task status monitoring.